Musubi Japan

Laboratorio: arquitectura, subarquitectura, SOLID y patrones

Ingeniería de Software II (2016702)

Daniel Gracia

Octubre de 2026

1. Fase 1. Dominio y problema

1.1. Sector profesional

El sector es la educación de un segundo idioma. Dentro de él, el japonés se distingue del español porque la escritura, el léxico y la gramática no se recorren con un solo alfabeto. El hiragana y el katakana son silabarios: permiten leer una frase en voz alta sin conocer el significado de las palabras. El kanji y el vocabulario cargan ese significado, y para leer y escribir un texto común hace falta conocer más de 2000 caracteres. La gramática, además, sigue una lógica distinta de la de las lenguas romances, y eso pesa cuando quien estudia tiene el español como primera lengua. Esas partes se enseñan y se practican a menudo por separado, aunque en un texto aparecen juntas. El avance no se mide solo con palabras aisladas: reconocer una pieza es distinto de usarla bien en un contexto.

1.2. Actores

- **Aprendiz de japonés.** Quien estudia el idioma en el celular, sea novato o ya tenga conocimientos previos, y busca una herramienta de reforzamiento.
- **Proveedor de IA.** Sistema externo de la capa de acompañamiento. Sostiene a la mascota cuando hay red y el aprendiz tiene acceso a esa capa.
- **Red de anuncios y pasarela de pago.** Sistemas externos que financian la capa de IA, por anuncios o por una opción de pago. El estudio sin mascota no los usa.

1.3. Problema

El refuerzo del japonés está repartido en varias aplicaciones. Unas trabajan el kanji, otras el vocabulario y la gramática, otras los textos y las lecturas. Las que reúnen varios de esos componentes tienen un costo alto. Quien estudia salta de una herramienta a otra, y el kanji, la palabra, la gramática y el escrito quedan sueltos.

Musubi Japan reúne esos contenidos en un solo lugar y los entrega con la aplicación. El recorrido lo decide el aprendiz: sigue el camino establecido por la aplicación o incorpora, poco a poco, las frases de lo que va aprendiendo, en el orden que le sirve. Cada frase entra como material y la aplicación la desglosa en vocabulario, reglas gramaticales y kanji. Cada pieza guarda dos grados de avance: el reconocimiento, cuando la persona la identifica, y la interiorización, cuando ya la usa bien.

El ejercicio mueve solo las piezas que pone en juego. Dibujar el kanji de 食べる (食, en たべる, con el sentido general de comida) actualiza ese kanji. Producir りんごを食べます (como una manzana) actualiza las palabras りんご (manzana) y 食べる (comer), la partícula を (la acción recae sobre el objeto directo), el verbo en presente (食べます, forma cordial) y el kanji de 食べる.

El estudio cabe entero en el teléfono, sin red y sin mascota. En modo offline la interiorización es autoevaluativa y no hay anuncios. La mascota vive en otra capa. Cuando está disponible, acompaña el aprendizaje con contexto de uso cotidiano: cuándo conviene una frase y cuándo otra, o cuando conviene usar una palabra en un contexto diferente.

1.4. Requisitos funcionales

RF1. Como aprendiz novato, quiero seguir la ruta de aprendizaje estándar que ofrece la aplicación para adentrarme en el aprendizaje del japonés.

RF2. Como aprendiz, quiero incorporar en mi orden las frases que voy aprendiendo para no olvidarlas y poder usarlas más adelante.

RF3. Como aprendiz, quiero que se registre en cada pieza (vocabulario, kanji o gramática) el reconocimiento y la interiorización por separado para saber si ya la identifico o ya la uso.

RF4. Como aprendiz, quiero practicar una pieza aislada (vocabulario, kanji o gramática) para entender su significado y en qué otras frases puede usarse.

RF5. Como aprendiz, quiero que, al practicar una frase, el progreso de sus piezas (vocabulario, kanji y gramática) avance para dedicar menos tiempo a cada una.

RF6. Como aprendiz, quiero ver el progreso de mi avance general para saber qué tanto he avanzado en mi aprendizaje.

RF7. Como aprendiz, quiero acceder a lecturas en japonés para practicar lo que he aprendido y fortalecer mi aprendizaje del idioma.

RF8. Como aprendiz, quiero usar la aplicación sin conexión para no depender de los anuncios ni del uso obligatorio de la mascota.

RF9. Como aprendiz, quiero que la mascota me dé ejercicios nuevos e información de uso correcto para fortalecer mi uso del japonés.

RF10. Como sistema, quiero ofrecer la mascota mediante anuncios o pago para sostener su servicio.

1.5. Atributos de calidad

Disponibilidad. El aprendiz solicita la ruta, la práctica, las lecturas o el avance sin conexión; el sistema los entrega en el dispositivo en el 100 % de las solicitudes.

Rendimiento. El aprendiz termina de practicar una frase; el sistema muestra el progreso de sus piezas y el avance general en menos de un segundo.

Mantenibilidad. Los componentes de la aplicación permanecen desacoplados; cada componente y cada conexión entre ellos se prueba sin ejecutar el resto del sistema.

Seguridad. Si el aprendiz estudia sin conexión, el sistema conserva sus frases y su avance solo en el teléfono. Si usa la mascota, el sistema envía al proveedor únicamente la solicitud, sin información personal.

2. Fase 2. Arquitectura macro

Musubi Japan usa un monolito modular en el teléfono y un solo servicio para la mascota, desplegado en un servidor. La práctica, las lecturas, el detalle, el avance y el almacén local viven en la aplicación, junto con los grupos de frases. La ruta sugerida es el grupo que ofrece el sistema, ordenado por temas. El aprendiz puede seguirlo, crear sus propios grupos con las frases que considera relacionadas, o pasar de uno al otro. Al guardar una frase, una función del grupo la desglosa en hiragana, katakana, kanji, vocabulario y gramática, y asocia las piezas de ese grupo. Si una pieza ya trae avance de otro grupo, ese progreso sirve para enfocarse en lo que falta. El avance muestra el progreso general, luego cada tipo de pieza y, dentro, cómo va cada una. Desde una lectura o desde el avance, un elemento abre el detalle de su pieza: ahí se entiende y, si el aprendiz quiere, se incorpora a un grupo. El módulo de la mascota queda aparte del estudio. Con la respuesta del proveedor incorpora frases nuevas en un grupo, propone sesiones en la práctica y genera lecturas. Con red abre los anuncios o el pago y envía solo la solicitud al servicio de la mascota, que la reenvía al proveedor de IA. Ese servicio no guarda datos del aprendiz. El almacén local, cuando hay red, deja una copia de seguridad en el Google Drive del aprendiz. No hay gateway ni broker.

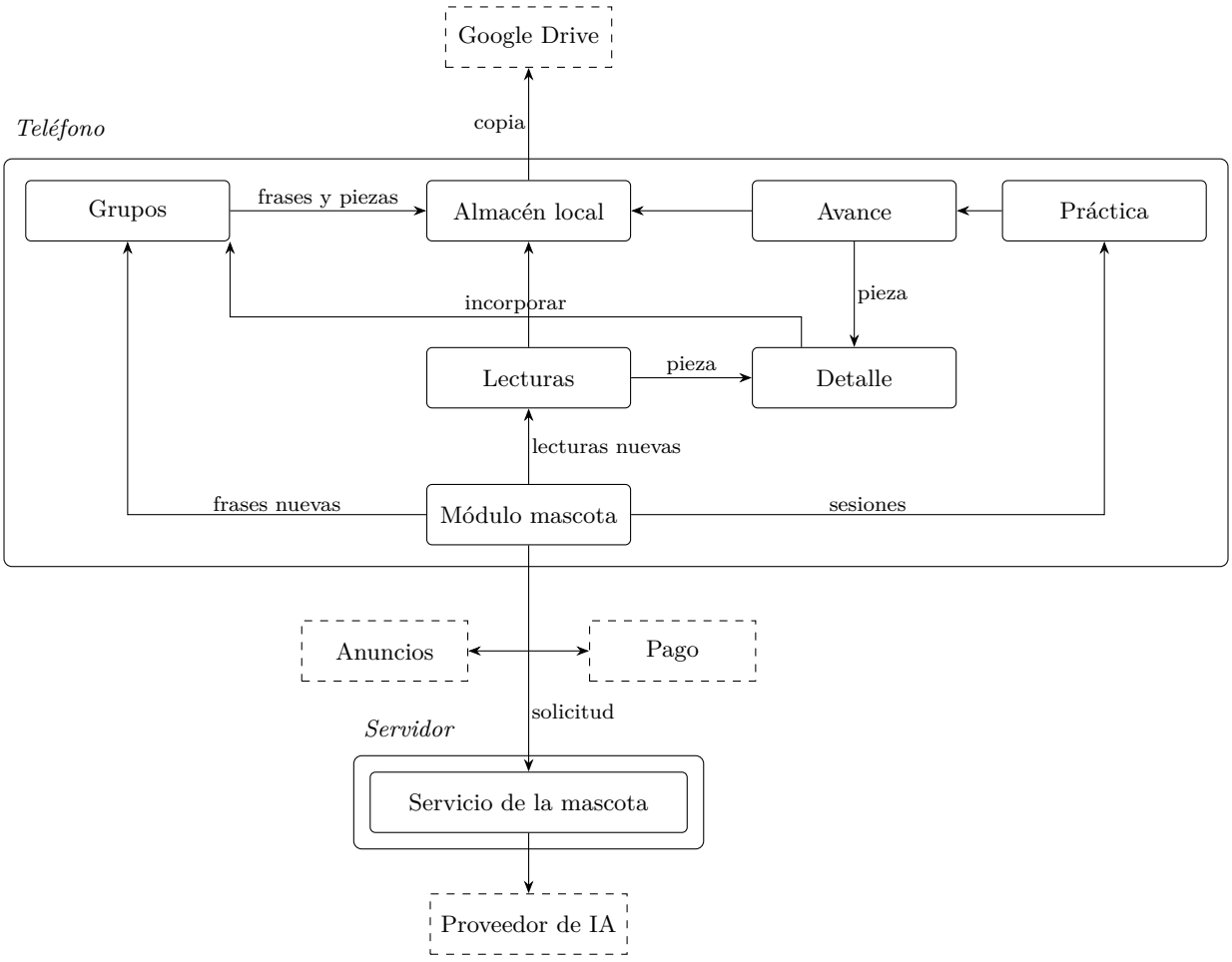


Figura 1. Arquitectura macro de Musubi Japan. Trazo continuo: componentes de Musubi Japan. Trazo punteado: sistemas externos.

2.1. Justificación frente a microservicios

Partir la ruta, la práctica, las lecturas y el avance en microservicios no cumple los escenarios de la fase 1.

Disponibilidad. Esas funciones responden sin red en el 100 % de las solicitudes. En servicios separados dejan de estar en el teléfono.

Rendimiento. El progreso de una frase se muestra en menos de un segundo, sin esperar a la red. Un servicio remoto no cumple esa medida.

Seguridad. Sin conexión, las frases y el avance se quedan en el teléfono. La única copia que sale, cuando hay red, es la del Google Drive del aprendiz. El servidor de la mascota no la guarda.

Mantenibilidad. Cada componente y cada conexión se prueban sin ejecutar el resto del sistema. Esa separación es entre módulos de la misma aplicación.

El servicio de la mascota sí se despliega aparte, en un servidor: la clave del proveedor no va en la aplicación, y los anuncios o el pago sostienen ese servicio. Sigue siendo un solo servicio.

3. Fase 3. Componente y subarquitectura

3.1. Componente elegido

Práctica es el componente que se diseña porque ahí el aprendiz repasa lo que ya vio y, cuando hay conexión, construye con más libertad. Los grupos se limitan a ordenar frases y a desglosarlas; Avance guarda las marcas y las muestra; Lecturas y Detalle enseñan la pieza. La mascota no reemplaza esa regla: llama la herramienta de Práctica y ofrece la sesión.

Repasar un kanji o un vocabulario es un ejercicio acotado. El aprendiz lo reconoce, lo identifica y lo escribe, a mano o con el teclado, y ese repaso mueve solo esa pieza. Repasar gramática es construir frases. Sin conexión, las frases tienen que ser las que ya se hicieron antes, de modo que la aplicación las contraste con lo guardado. Con conexión, el agente puede evaluar construcciones que el aprendiz no había hecho.

El modo libre extiende esa construcción. El aprendiz arma frases o párrafos, y el agente los evalúa y los corrige. Hace falta la conexión, porque sin el agente no hay con qué juzgar un texto nuevo. La interiorización queda solo en las piezas que el aprendiz usó bien en ese contexto. Las demás reciben reconocimiento cuando la forma es correcta, pero no encaja en el contexto pedido. Un hiragana o un katakana ya interiorizado no vuelve a ese foco: el repaso y la corrección se concentran en el kanji, el vocabulario y la gramática que todavía lo necesitan.

3.2. MVVM

La subarquitectura es MVVM porque la pantalla cambia de ejercicio y la regla no. Reconocer, identificar o escribir un kanji no se decide en la vista, y la corrección de un párrafo tampoco. El modelo no sabe si la sesión la abrió el aprendiz o la mascota.

Vista. `PantallaDePractica` muestra el repaso o la construcción y recoge lo que el aprendiz escribe, a mano o con el teclado. Envía esa respuesta y no marca reconocimiento ni interiorización.

ViewModel. `SesionDePractica` abre tres sesiones. El repaso de kanji o vocabulario pide reconocer, identificar y escribir esa pieza. La gramática sin conexión construye una frase ya guardada. La sesión libre, que es la herramienta de la mascota, deja armar frases o párrafos para que el agente los evalúe. El ViewModel lee en el almacén lo que el modo necesita, pide el resultado al modelo y lo registra en Avance.

Modelo. `Sesion`, `Pieza`, `Frase` y `ReglaDeEfecto` separan el repaso de la construcción. El repaso de una pieza mueve solo esa pieza. La frase ya hecha se contrasta con lo guardado. En el modo libre, la interiorización queda en las piezas usadas bien en el contexto, y el reconocimiento en las que tienen la forma correcta fuera de él. El kana ya interiorizado no entra.

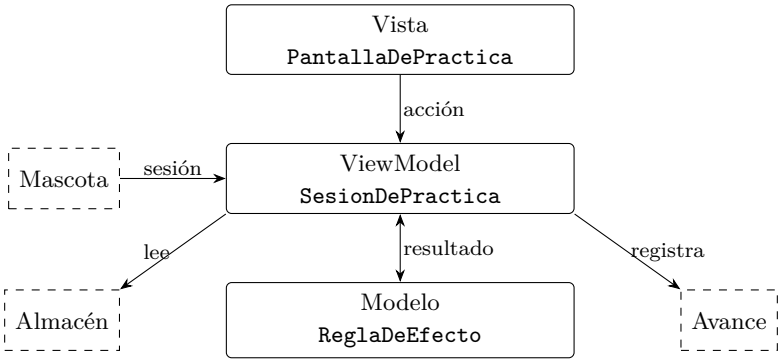


Figura 2. MVVM de Práctica. Trazo punteado: queda fuera del componente.

4. Fase 4. SOLID

El diseño de Práctica separa la sesión, el modo de ejercicio y la regla que reparte las marcas. La figura 3 muestra esas clases. En responsabilidad única y en abierto/cerrado se indica el diseño anterior, que mezclaba razones de cambio.

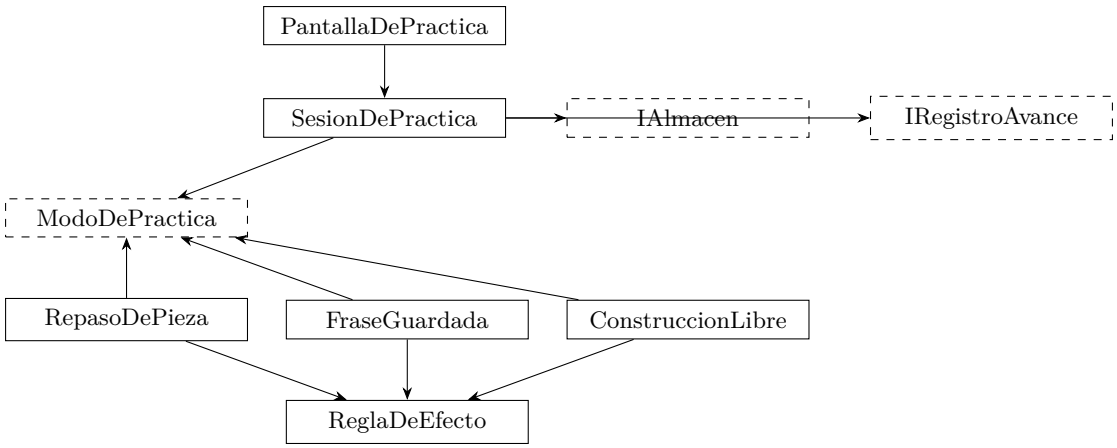


Figura 3. Clases de Práctica. Trazo punteado: interfaz.

Responsabilidad única. `SesionDePractica` cambia si cambia el flujo de la sesión: qué ejercicio se abre y cuándo se lee el almacén. `ReglaDeEfecto` cambia si cambia el reparto entre reconocimiento e interiorización. Antes las dos razones vivían en la sesión, y agregar el modo libre obligaba a modificar también el cálculo de las marcas.

Abierto/cerrado. `SesionDePractica` depende de `ModoDePractica`. `RepasoDePieza`, `FraseGuardada` y `ConstruccionLibre` son implementaciones, y un modo nuevo entra como otra clase. Antes un condicional dentro de la sesión elegía el ejercicio, así que cada modo modificaba esa clase.

Sustitución de Liskov. `RepasoKanji` y `RepasoVocabulario` sustituyen a `RepasoDePieza`: reconocen, identifican y escriben, y devuelven el avance de una sola pieza. `ConstruccionLibre` no es un subtipo de ese repaso, porque necesita al agente y puede marcar varias piezas.

Segregación de interfaces. `IRepaso` expone reconocer, identificar y escribir. `IConstruccion` expone construir la frase. `IEvaluacion` expone la corrección del agente. Antes una sola interfaz juntaba las tres, y el repaso de una pieza implementaba la evaluación sin usarla.

Inversión de dependencias. `SesionDePractica` recibe `IAlmacen` e `IRegistroAvance`. No construye el almacén local ni el componente Avance. Antes el ViewModel creaba esas clases concretas y la sesión no se podía probar sin ellas.

5. Fase 5. Patrones de diseño

Los seis patrones resuelven situaciones que Práctica ya tiene: abrir el modo sin construirlo dentro de la sesión, no alterar la frase guardada, escribir por trazo o por teclado, unificar la corrección local y la del agente, repartir las marcas y filtrar el modo libre. En cada ficha se nombra la alternativa que se dejó de lado.

5.1. Factory Method

Problema. La sesión abre un repaso de kanji, un repaso de vocabulario, una frase ya guardada o la construcción libre. Si `SesionDePractica` hace el `new`, cada modo nuevo vuelve a modificarla.

Solución. `FabricaDeModo` declara la creación y la subclase devuelve el modo concreto. Se descartó Abstract Factory: no hay una familia de objetos que deba nacer junta, solo el modo que el aprendiz abrió.

Participantes. Creador: `FabricaDeModo`. Creadores concretos: `FabricaDeRepaso`, `FabricaDeFraseGuardada` y `FabricaDeConstruccion`. Producto: `ModoDePractica`. Productos concretos: `RepasoDePieza`, `FraseGuardada` y `ConstruccionLibre`.

SOLID. Abierto/cerrado e inversión de dependencias. La sesión recibe la fábrica y no conoce la clase concreta.

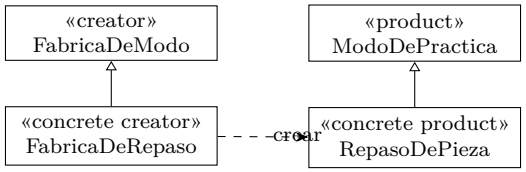


Figura 4. Factory Method al abrir un repaso. Las otras dos fábricas siguen el mismo esquema.

5.2. Prototype

Problema. Sin conexión, el repaso de gramática compara el intento con la frase guardada. Si la sesión escribe sobre ese objeto, la respuesta del almacén cambia.

Solución. `Frase` se clona y el intento usa la copia. Se descartó Builder: la frase no se arma con datos opcionales; ya está guardada y el ejercicio parte de ella.

Participantes. Prototipo: `Frase`. Cliente: `FraseGuardada`, que pide la copia. El original permanece en el almacén.

SOLID. Responsabilidad única. Guardar la frase y practicar el intento no comparten razón de cambio.

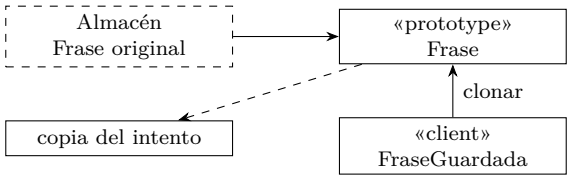


Figura 5. Prototype. La sesión practica sobre la copia.

5.3. Bridge

Problema. El repaso de kanji y el de vocabulario piden reconocer, identificar y escribir, y la escritura entra por trazo o por teclado. Una subclase por cada par multiplica las clases en cuanto aparece otra pieza o otra forma de capturar.

Solución. `RepasoDePieza` guarda una `Captura`, y las dos variaciones cambian por separado. Se descartó Adapter: el trazo y el teclado no son interfaces ajenas; son dos implementaciones propias de la misma captura.

Participantes. Abstracción: `RepasoDePieza`. Abstracciones refinadas: `RepasoKanji` y `RepasoVocabulario`. Implementador: `Captura`. Implementadores concretos: `CapturaAMano` y `CapturaTeclado`.

SOLID. Abierto/cerrado. Sustitución de Liskov: cualquiera de las dos capturas ocupa el lugar de `Captura`.

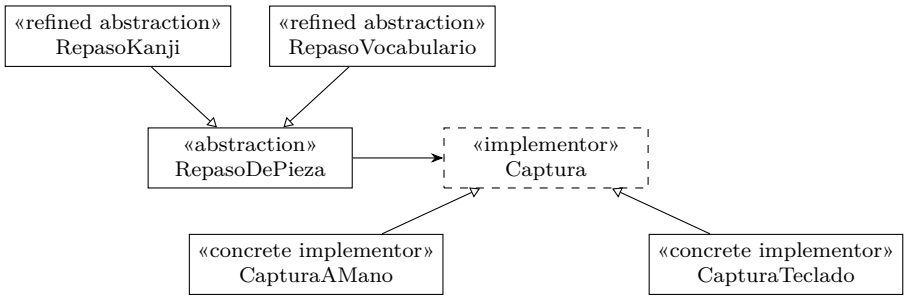


Figura 6. Bridge entre el tipo de pieza y la captura.

5.4. Adapter

Problema. Sin conexión, la frase se comprueba contra lo guardado. Con conexión, el agente devuelve la corrección en el formato del proveedor. El modo necesita un solo resultado: qué piezas entran, si la forma es correcta y si el uso cabe en el contexto pedido.

Solución. `IEvaluacion` es el destino común. Un adaptador envuelve la comparación del almacén y otro la respuesta del proveedor. Se descartó Facade: no hay varias clases de un subsistema que ocultar, sino una forma de resultado que no coincide con la otra.

Participantes. Destino: `IEvaluacion`. Adaptadores: `AdaptadorDeFraseGuardada` y `AdaptadorDelAgente`. Adaptados: la comparación del almacén y la respuesta del proveedor. Clientes: `FraseGuardada` y `ConstruccionLibre`.

SOLID. Inversión de dependencias y segregación de interfaces. Evaluar queda aparte de reconocer y de construir.

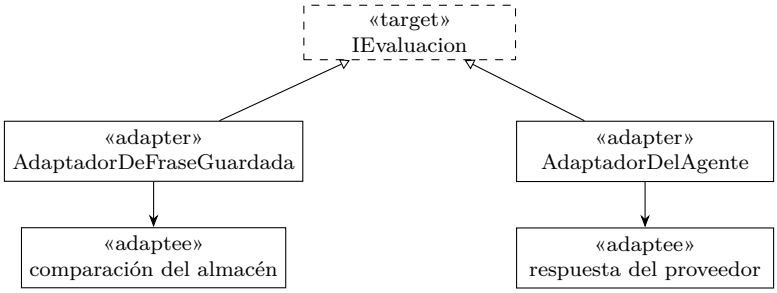


Figura 7. Adapter hacia `IEvaluacion`.

5.5. Strategy

Problema. El reparto no es el mismo en los tres modos. El repaso mueve una sola pieza. La frase guardada mueve las piezas que esa frase pone en juego. En el modo libre, la interiorización queda en las piezas usadas bien en ese contexto, y las demás reciben reconocimiento si la forma es correcta y el contexto no. Un condicional dentro de `ReglaDeEfecto` cambiaría con cada reparto nuevo.

Solución. `ReglaDeEfecto` delega en `PoliticaDeMarca`. Se descartó State: la política no cambia porque la sesión avance de etapa; queda elegida al abrir el modo.

Participantes. Contexto: `ReglaDeEfecto`. Estrategia: `PoliticaDeMarca`. Estrategias concretas: `EfectoDeUnaPieza`, `EfectoDeFrase` y `EfectoDeContexto`.

SOLID. Abierto/cerrado y responsabilidad única. Un reparto nuevo es una clase nueva y no reabre la sesión.

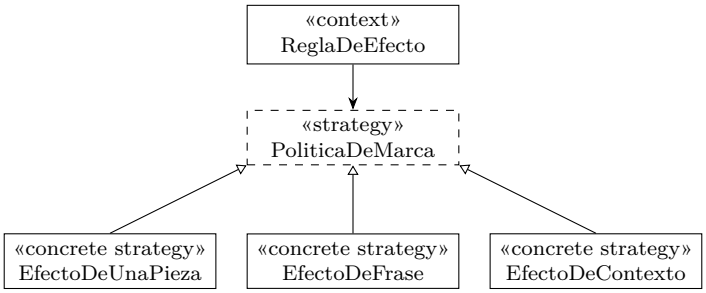


Figura 8. Strategy del reparto entre reconocimiento e interiorización.

5.6. Chain of Responsibility

Problema. Antes de aplicar `EfectoDeContexto`, el modo libre aparta el hiragana y el katakana ya interiorizados, y separa la forma correcta del uso en el contexto pedido. Si esos filtros viven en la política, un filtro nuevo la modifica.

Solución. Cada filtro decide sobre la pieza o la pasa al siguiente. Se descartó Template Method: la cantidad y el orden de los filtros cambian, y no son pasos fijos de una misma receta.

Participantes. Manejador: `FiltroDeMarca`. Manejadores concretos: `FiltroDeKanaDominado`, `FiltroDeForma` y `FiltroDeContexto`. Cliente: `ConstruccionLibre`, que arma la cadena y entrega lo que queda a `EfectoDeContexto`.

SOLID. Abierto/cerrado y responsabilidad única. Cada filtro tiene una sola decisión.

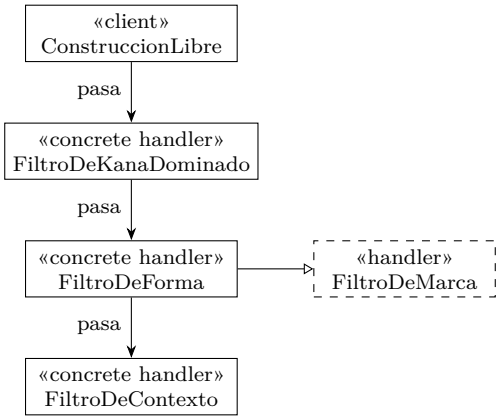


Figura 9. Cadena de filtros del modo libre, antes de `EfectoDeContexto`.